



Rapport de Projet

Software Engineering

Hugo CLAVEILLE

Master 1 Informatique

30 octobre 2025

Table des matières

1	Introduction	3
2	Présentation du problème et des objectifs	4
3	Conception et Implémentation	5
3.1	Structure générale du projet	5
3.2	Methodes de compression	5
3.2.1	Méthode 1 : Entiers scindables	5
3.2.2	Méthode 2 : Entiers non scindables	6
3.3	Gestion des débordements	6
3.3.1	Méthode 1	6
3.3.2	Méthode 2	7
4	Résultats et Analyse des Benchmarks	8
4.1	Méthodologie de Benchmark	8
4.2	Résultats Expérimentaux	8
4.2.1	Temps de compression et de décompression	9
4.2.2	Temps d'accès direct (get)	9
4.3	Resultats détaillés par distribution et nombre de bits	10
4.3.1	Nombre de bits	10
4.4	Visualisations et Interprétation	11
4.5	Discussion	11

5	Benchmark	13
6	Conclusion et Perspectives	14
7	Résultats expérimentaux	15
7.1	Temps de compression	15
7.2	Temps de décompression	15
7.3	Temps moyen d'accès (get())	15
7.4	Analyse des résultats	15
8	Résultats des Benchmarks	16
8.1	Temps de compression	16

1. Introduction

Dans le cadre de l'Unités d'Enseignement de Software Engineering-2025, il nous a été demandé de réaliser un projet individuel visant à étudier et implémenter des méthodes de transmission optimisée de données. La transmission de tableaux d'entiers constitue un problème fondamental dans de nombreux systèmes informatiques et réseaux, en particulier pour les applications nécessitant un échange rapide et efficace de grandes quantités de données. Ce projet s'inscrit donc dans un contexte pratique où la performance et l'efficacité des transferts d'information sont essentielles.

L'objectif est d'étudier différentes méthodes de compression de ces tableaux afin de réduire le nombre de bits transmis tout en conservant un accès direct à chaque élément. La méthode choisie, appelée Bit Packing, représente chaque entier avec le minimum de bits nécessaires, regroupés ensuite dans des entiers standards pour la transmission. Deux variantes seront explorées : une autorisant l'écriture d'un entier compressé sur deux entiers consécutifs, et une qui ne le permet pas. Le projet inclut également des benchmarks pour évaluer les gains de performance.

La gestion des débordements (overflow) est un aspect crucial de ce projet. En effet, lorsque les entiers à compresser dépassent la capacité de bits allouée, il est nécessaire de mettre en place une stratégie efficace pour gérer ces cas. La méthode choisie consiste à utiliser des entiers supplémentaires pour stocker les valeurs qui ne peuvent pas être représentées dans le format compressé initial. Cette approche permet de maintenir l'intégrité des données tout en optimisant l'espace de stockage.

2. Présentation du problème et des objectifs

L'objectif principal du projet est de concevoir et comparer deux méthodes de compression de tableaux d'entiers en utilisant le principe du *Bit Packing*. Chaque entier est représenté à l'aide du nombre minimal de bits nécessaire, puis compacté dans un tableau d'entiers 32 bits standard.

Deux variantes ont été implémentées :

- **Méthode 1** : un entier compressé peut être réparti sur deux entiers consécutifs.
- **Méthode 2** : un entier compressé doit être contenu dans un seul entier.

Le but de ces deux approches est de déterminer laquelle offre le meilleur compromis entre temps de compression, temps de décompression et accessibilité directe à un élément donné. Des benchmarks automatisés ont été mis en place pour comparer ces deux méthodes sur des tailles de tableaux croissantes, distributions différentes ainsi que différents niveaux de compression (nombre de bits par entier).

3. Conception et Implémentation

3.1 Structure générale du projet

Le projet est structuré autour de trois composants principaux :

- Les classes `Method1Compressor` et `Method2Compressor` implémentant les deux méthodes de compression ;
- Un module de **benchmark** mesurant les performances pour différentes tailles de tableaux et valeurs de `maxbit` ;
- Un générateur de données permettant de produire des listes d'entiers selon plusieurs distributions (aléatoire uniforme, croissante, bruitée...).

3.2 Methodes de compression

Les deux classes `Method1Compressor` et `Method2Compressor` héritent d'une classe de base `CompressorBase` qui définit l'interface commune pour la compression et la décompression. Chaque classe implémente les méthodes `compress()`, `decompress()` et `get()` selon la logique spécifique à chaque méthode.

Les deux méthodes utilisent des algorithmes différents pour gérer la répartition des bits et l'accès aux éléments compressés que nous détaillerons ci-dessous. Nous traiterons plus tard de la gestion des débordements (overflow) lorsque les entiers à compresser dépassent la capacité allouée. Chaque entier compressé utilisant un nombre de bits inférieur ou égal à `maxbit` est annoté d'un préfixe 0 avant sa représentation binaire. Si l'entier dépasse cette capacité, il est annoté d'un préfixe 1 et stocké dans une structure dédiée aux débordements, décrite dans la section suivante.

3.2.1 Méthode 1 : Entiers scindables

Dans cette méthode, un entier compressé peut être réparti sur deux entiers consécutifs. Cela permet une utilisation plus efficace de l'espace lorsque les entiers à compresser ne s'alignent pas parfaitement avec les limites des entiers 32 bits. Par exemple, si chaque entier compressé utilise 10 bits, trois entiers compressés peuvent être stockés dans un

entier 32 bits, et le quatrième sera scindé entre le premier et le deuxième entier suivant. (voir Figure 3.1) Cette approche maximise l'utilisation de chaque bit disponible, mais complique légèrement la logique d'accès aux éléments compressés.

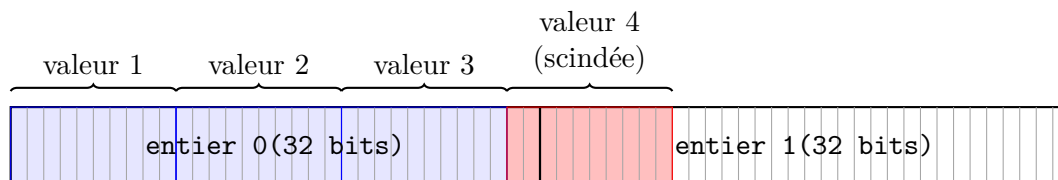


FIGURE 3.1 – Méthode 1 — un entier peut être scindé entre deux mots (ici : 2 + 8 bits).

3.2.2 Méthode 2 : Entiers non scindables

Dans cette méthode, chaque entier compressé doit être entièrement contenu dans un seul entier 32 bits. Cela simplifie l'accès aux éléments compressés, car chaque entier peut être récupéré directement sans avoir à gérer la logique de scission entre deux entiers. Cependant, cette approche peut entraîner une utilisation moins efficace de l'espace, car certains bits dans les entiers 32 bits peuvent rester inutilisés si le nombre de bits par entier compressé ne s'aligne pas parfaitement avec 32.

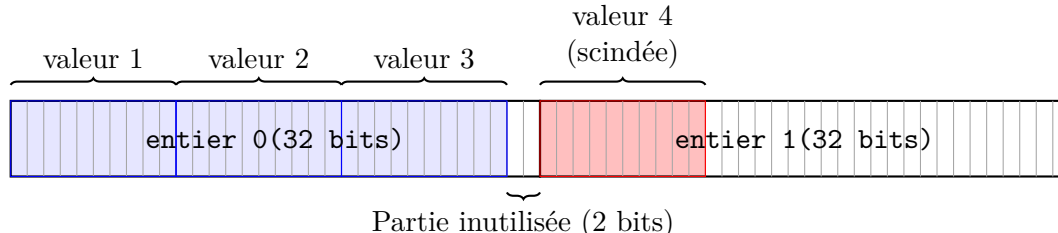


FIGURE 3.2 – Méthode 1 — un entier peut être scindé entre deux mots (ici : 2 + 8 bits).

3.3 Gestion des débordements

A FAIRE ————— La gestion des débordements est gérée différemment selon la méthode choisie, mais utilisent le même principe de détection. Lorsqu'un entier à compresser dépasse la capacité maximale définie par `maxbit`, on stocke son indice

3.3.1 Méthode 1

Lorsqu'un entier dépasse la taille maximale définie par `maxbit`, on le stocke en fin de chaîne dans un tableau dédié aux débordements. Un indicateur spécial est utilisé dans le

flux compressé pour signaler la présence d'un débordement. Lors de la décompression, si cet indicateur est rencontré, la valeur correspondante est récupérée depuis le tableau des débordements.

3.3.2 Méthode 2

4. Résultats et Analyse des Benchmarks

4.1 Méthodologie de Benchmark

Afin d'évaluer les performances des deux méthodes de compression développées, une campagne de mesures automatisées a été réalisée à l'aide d'un script Python dédié. Chaque configuration est définie par quatre paramètres principaux :

- le nombre de bits utilisés pour encoder chaque entier (`max_bits`);
- la taille du tableau d'entrée (`n`);
- la méthode de compression (`method1` ou `method2`).
- le type de distribution des entiers (`aléatoire uniforme`, `croissante` ou `bruitée`).

Pour chaque combinaison de paramètres, plusieurs itérations sont effectuées afin d'obtenir des moyennes représentatives des temps de traitement. Les trois opérations principales sont mesurées séparément :

- **Compression** : transformation du tableau d'entiers bruts en format compressé.
- **Décompression** : restitution du tableau original à partir du format compressé.
- **Accès direct** (`get`) : récupération aléatoire d'éléments sans décompression complète.

Chaque test est répété plusieurs fois pour éliminer les effets de cache ou d'aléa d'exécution. Les résultats sont sauvegardés dans un fichier CSV brut, puis agrégés en moyennes pour faciliter l'analyse et la génération de graphiques.

4.2 Résultats Expérimentaux

Après avoir exécuté les benchmarks selon la méthodologie décrite précédemment, nous pouvons maintenant nous intéresser aux résultats obtenus. Dans un premier temps, nous allons examiner les temps de compression et de décompression ainsi que les temps d'accès direct aux éléments compressés moyen pour chaque méthode sans tenir compte des différentes distributions de données ni du nombre de bits d'encodage.

4.2.1 Temps de compression et de décompression

Les temps de compression augmentent naturellement avec la taille du tableau et la complexité du codage (nombre de bits utilisés). Mais, malgré ce qu'on pourrait attendre, la **Méthode 1** montre des performances nettement supérieures à la **Méthode 2** pour des tailles de tableaux plus importantes. Cela s'explique par la flexibilité offerte par la possibilité de scinder les entiers entre deux mots, ce qui optimise l'utilisation de l'espace et réduit le nombre d'opérations nécessaires. Voir figure 4.1.

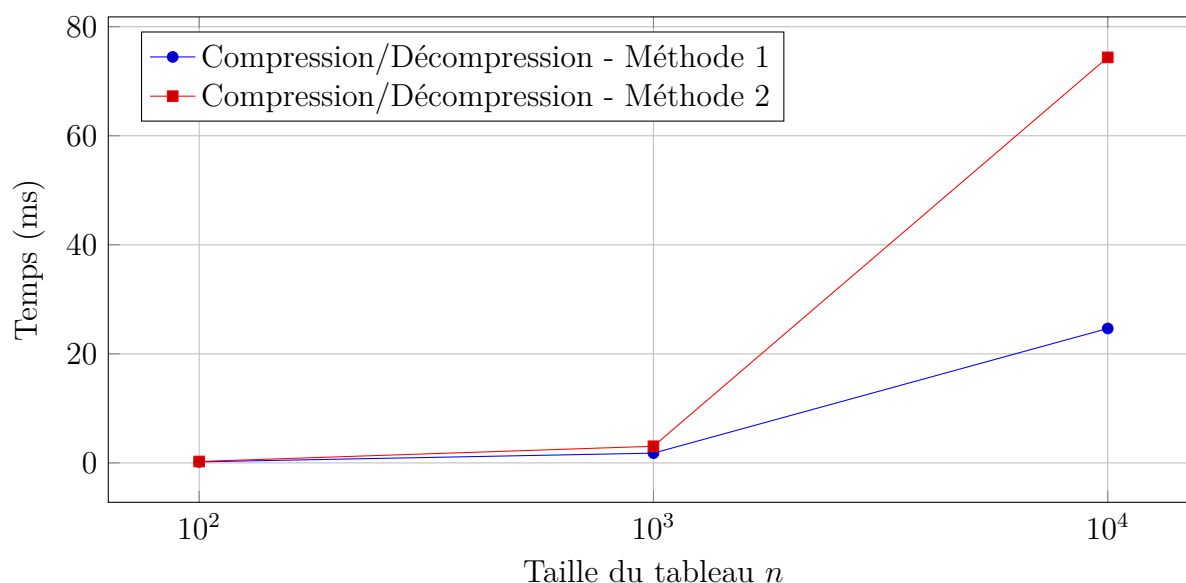


FIGURE 4.1 – Performance de la Méthode 1 : temps de compression et décompression moyen en fonction de la taille du tableau.

4.2.2 Temps d'accès direct (get)

La mesure des accès aléatoires est clairement étrange, le temps moyen d'accès direct via la méthode `get()` semble diminuer avec l'augmentation de la taille du tableau, ce qui est contre-intuitif. Cela pourrait être dû à la modification du nombre de bits utilisés pour l'encodage. Nous y reviendrons plus tard.

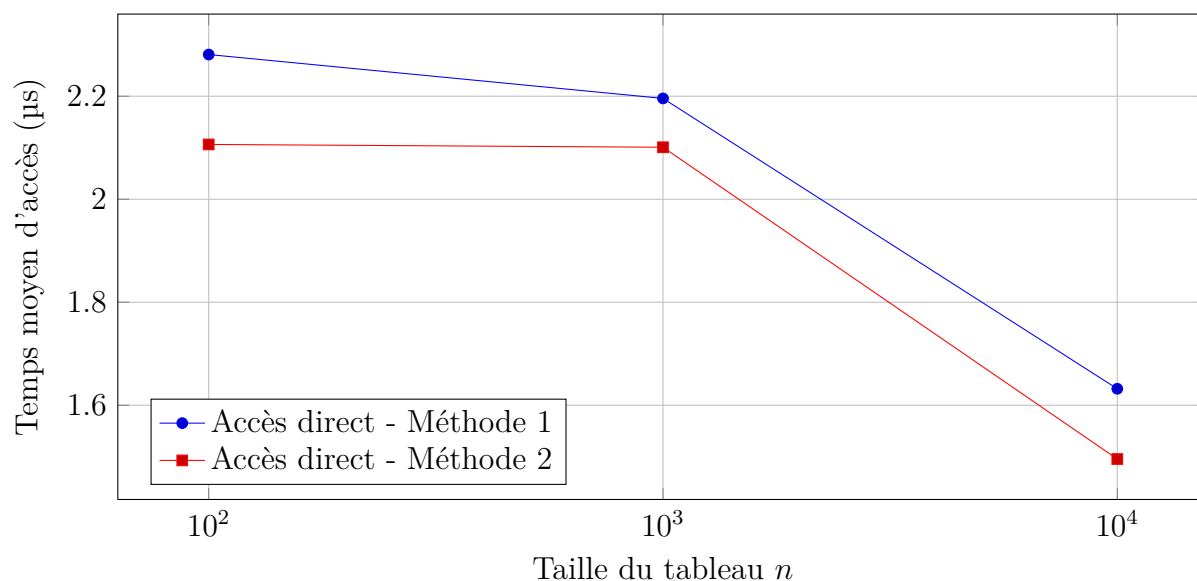


FIGURE 4.2 – Performance de la Méthode 1 : temps moyen d'accès direct en fonction de la taille du tableau.

4.3 Résultats détaillés par distribution et nombre de bits

Afin de mieux comprendre les performances des deux méthodes, nous avons analysé les résultats en fonction des différentes distributions de données et du nombre de bits utilisés pour l'encodage.

4.3.1 Nombre de bits

L'impact du nombre de bits sur les performances est significatif. En général, une augmentation du nombre de bits utilisés pour l'encodage conduit à une amélioration des temps de compression et de décompression, mais peut également augmenter le temps d'accès direct. Les graphiques suivants illustrent ces variations en fonction de la taille du tableau et de la méthode utilisée.

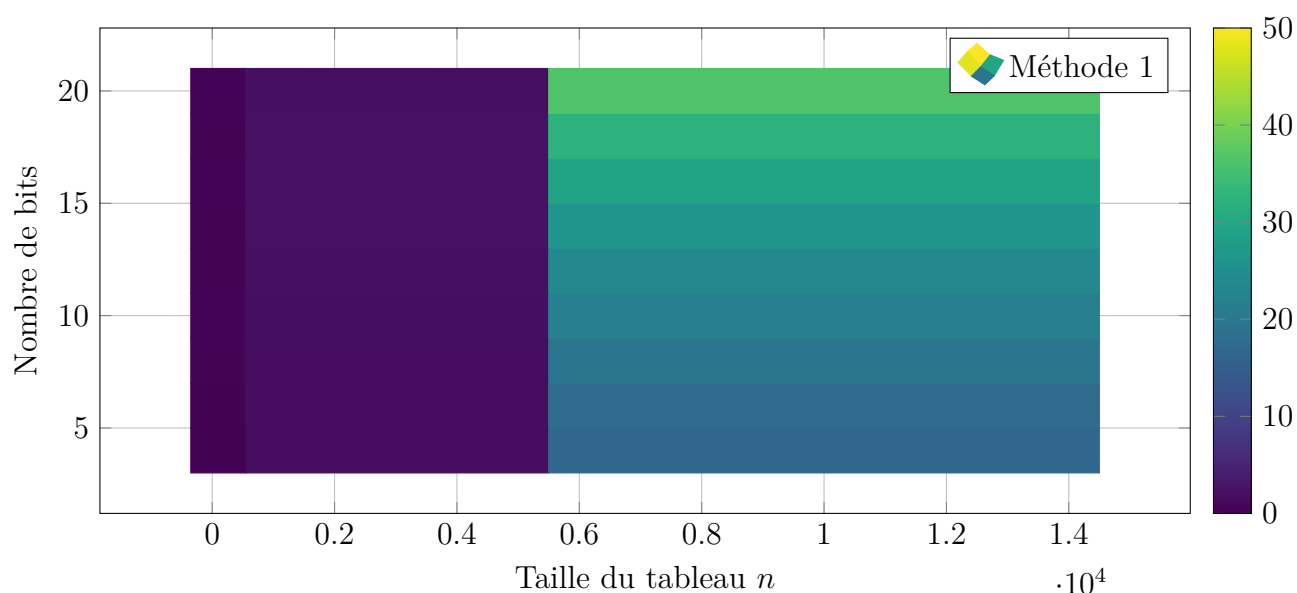


FIGURE 4.3 – Heatmap des performances en fonction de la taille du tableau et du nombre de bits utilisés.

4.4 Visualisations et Interprétation

Les graphiques 2D permettent une lecture intuitive de l'évolution des performances pour chaque paramètre. Afin de mieux comprendre les interactions entre la taille du tableau, le nombre de bits et le temps d'exécution, des **graphiques 3D** et **heatmaps** ont été utilisés. Ces visualisations offrent une représentation synthétique des zones de performance optimale, et mettent en évidence les compromis entre compacité et vitesse.

Par exemple, les heatmaps montrent clairement que la **Méthode 2** est plus efficace pour des tailles modérées et un encodage limité (moins de 12 bits), tandis que la **Méthode 1** devient plus intéressante lorsque la taille des données et le nombre de bits augmentent.

4.5 Discussion

L'analyse des résultats suggère que le choix entre les deux méthodes dépend du contexte d'utilisation :

- pour des tableaux de taille moyenne et un besoin de vitesse, la **Méthode 2** est à privilégier ;
- pour des grands volumes ou des tailles de mots variables, la **Méthode 1** offre un meilleur compromis entre compacité et flexibilité.

L'intégration d'une stratégie adaptative, sélectionnant automatiquement la méthode la plus appropriée selon le profil des données, pourrait constituer une amélioration future

du système.

5. Benchmark

Les benchmarks ont été réalisés automatiquement pour des tailles de tableau $n = 100, 1000$ et 10000 et des valeurs de `maxbit` comprises entre 4 et 20 (pas de 2). Chaque exécution mesure :

- le temps de compression,
- le temps de décompression,
- le temps moyen d'accès via la méthode `get()`.

Les résultats sont enregistrés dans un fichier JSON et agrégés dans un CSV pour exploitation graphique.

6. Conclusion et Perspectives

Ce projet a permis de comparer deux méthodes de compression d'entiers basées sur le principe du *Bit Packing*. Les résultats des benchmarks montrent que le compromis entre performance et complexité dépend fortement du contexte d'utilisation.

Les perspectives d'amélioration incluent :

- l'utilisation de parallélisme pour accélérer la compression,
- l'intégration d'un mode adaptatif choisissant automatiquement la méthode la plus efficace,
- l'ajout d'un module de visualisation temps réel des performances.

7. Résultats expérimentaux

7.1 Temps de compression

7.2 Temps de décompression

7.3 Temps moyen d'accès (`get()`)

7.4 Analyse des résultats

Les résultats montrent que la méthode 1, bien que plus coûteuse en mémoire dans certains cas, offre de meilleures performances en compression et en décompression lorsque la taille du tableau augmente. La méthode 2 reste plus stable, mais devient moins efficace pour de grands volumes de données.

8. Résultats des Benchmarks

8.1 Temps de compression

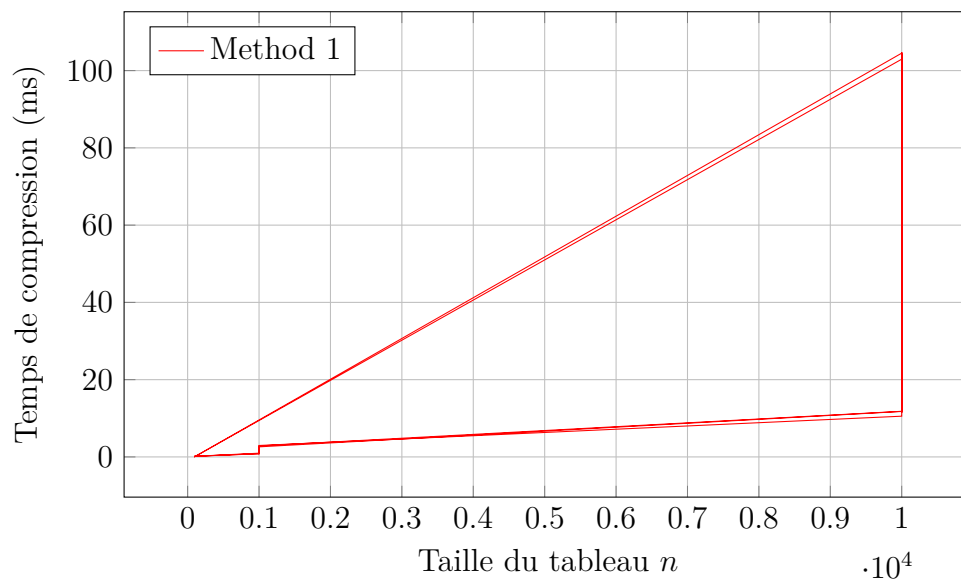


FIGURE 8.1 – Comparaison des temps de compression entre les deux méthodes selon la taille du tableau.